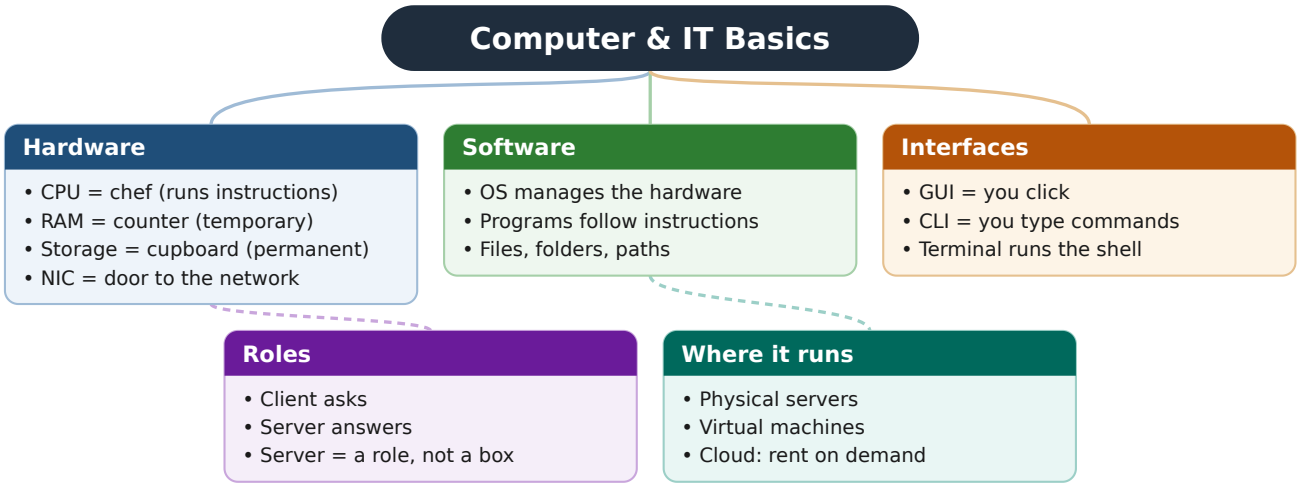
1. Computer & IT Basics

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 1 of 18 · Thirupathi Keloth · v2.0

Contents

1.0 Why this topic matters
1.1 What a computer does
1.2 Hardware components in depth
1.3 Units and number systems
1.4 Operating systems
1.5 Software
1.6 Files, folders and paths
1.7 The command line
1.8 Client–server model and how the web works
1.9 Networks and the internet (overview)
1.10 Data centers, virtualization and cloud
1.11 Security fundamentals
1.12 IT roles and how they work together
1.13 Basic troubleshooting mindset
1.14 Interview questions and answers
1.15 Self-check: I can...



1.0 Why this topic matters

Everything in DevOps and cloud runs on computers: physical servers in data centers, virtual machines, containers. When you choose an EC2 instance type, size a Kubernetes pod, debug a slow server or explain why a disk is full, you use the concepts in this topic. Learn them well once; every later topic builds on them.

1.1 What a computer does

```
INPUT ──► PROCESSING ──► OUTPUT
(keyboard,    (CPU + RAM)      (screen, network,
 network,          │             disk, speakers)
 disk)             ▼
              STORAGE (keeps data when power is off)
```

A computer runs **programs** (lists of instructions) on **data**. The CPU executes instructions; RAM holds the program and data being worked on; storage keeps everything permanently; the network connects it to other computers.

1.2 Hardware components in depth

CPU (Central Processing Unit)

Term	Meaning	Why it matters in DevOps/cloud
Core	An independent processing unit inside the CPU	More cores = more work in parallel (web servers, builds)
Thread / hyper-threading (SMT)	One core runs two instruction streams	Cloud **vCPU** = usually one hardware thread, not a full core
Clock speed (GHz)	Instruction cycles per second	Affects single-threaded performance
Cache (L1/L2/L3)	Very fast memory inside the CPU	Big caches help databases and compute-heavy work
Architecture	**x86-64** (Intel/AMD) or **ARM64** (AWS Graviton, Apple M-series, phones)	Software and container images must be built for the right architecture (`amd64` vs `arm64`)

CPU usage (%) shows how busy the cores are. **Load average** (Linux) shows how many tasks are running or waiting.

Memory (RAM)

- Fast, temporary working space; contents are lost when power goes off.
- Programs must be loaded into RAM to run.
- When RAM is full, the OS uses **swap** (disk as slow overflow memory) or kills processes (Linux **OOM killer**).
- Measured in GB/GiB. In the cloud you pick RAM per instance type (e.g. 1 GiB on `t3.micro`, 8 GiB on `m7i.large`) or per container/pod (memory requests/limits).

Storage

Type	How it works	Speed	Use
HDD (hard disk drive)	Spinning magnetic platters + moving head	Slow random access, cheap	Archives, backups, bulk data
SSD (SATA)	Flash memory, no moving parts	Fast	General use
NVMe SSD	Flash on the PCIe bus	Very fast	Databases, modern servers, cloud instance storage
Network/block storage	Disk provided over the network (e.g. AWS EBS)	Depends on configuration (IOPS, throughput)	Cloud servers
Object storage	Files stored via HTTP APIs (e.g. S3)	High throughput, higher latency per request	Backups, static files, data lakes

Performance terms: IOPS (input/output operations per second: many small reads/writes, e.g. databases), **throughput** (MB/s: large sequential reads/writes, e.g. video, logs), **latency** (time per operation, ms).

Other components

Component	Role
Motherboard	Connects everything (CPU socket, RAM slots, PCIe, storage ports)

Component	Role
GPU	Massively parallel processor: graphics, machine learning, video encoding (cloud: GPU instances)
NIC (network interface card)	Connects to the network; has a MAC address; speeds 1/10/25/100 Gbps
PSU	Power supply; servers have **redundant** PSUs
BIOS/UEFI firmware	First code that runs at power-on; tests hardware and starts the bootloader

Servers vs personal computers

Servers are built for **reliability and remote management**: redundant power supplies and disks (RAID), ECC memory (detects/corrects memory errors), many cores and lots of RAM, out-of-band management (iLO/iDRAC/IPMI: manage the machine even when the OS is down), rack-mounted (1U/2U), no monitor/keyboard. They usually run Linux without a graphical interface.

1.3 Units and number systems

Data units

Unit	Size
Bit (b)	0 or 1
Byte (B)	8 bits
Kilobyte KB / Kibibyte KiB	1,000 bytes / 1,024 bytes
MB / MiB	$1{,}000^2$ / $1{,}024^2$ bytes
GB / GiB	$1{,}000^3$ / $1{,}024^3$ bytes
TB / TiB	$1{,}000^4$ / $1{,}024^4$ bytes

Disk makers use decimal (GB); operating systems and Kubernetes often use binary (GiB, `Mi`, `Gi`). A "500 GB" disk shows as ~465 GiB.

Network speeds are in bits per second (Mbps, Gbps); **file sizes in bytes**. 100 Mbps ≈ 12.5 MB/s.

Number systems

System	Base	Digits	Where you see it
Decimal	10	0–9	Everyday numbers, IPv4 written form
Binary	2	0–1	How computers store everything; subnet masks
Hexadecimal	16	0–9, A–F	MAC addresses, IPv6, memory addresses, colors, hashes
Octal	8	0–7	Linux permissions (`chmod 755`)

Conversion methods are covered step by step in the Networking notes (binary and subnetting chapter).

Text encoding

Computers store text as numbers. **ASCII** covers English letters in 7 bits; **UTF-8** covers every language and emoji and is the standard everywhere (files, web, JSON). Mismatched encodings produce garbled characters; Windows vs Linux **line endings** (`\r\n` vs `\n`) break shell scripts edited on Windows (`dos2unix script.sh` fixes it).

1.4 Operating systems

An **operating system (OS)** manages hardware and gives programs a safe, shared way to use it.

OS responsibility	What it means
Process management	Starts/stops programs, shares CPU time between them (scheduling)
Memory management	Gives each process its own **virtual memory**, protects processes from each other, uses swap
File systems	Organizes data on disks into files and directories with permissions
Device management	Drivers let the OS talk to hardware
Networking	TCP/IP stack, sockets
Security	Users, permissions, isolation
User interface	Shell (CLI) and/or graphical desktop (GUI)

Kernel and user mode

The **kernel** runs with full hardware access (**kernel mode**). Normal programs run in **user mode** and must ask the kernel for anything privileged via **system calls** (read a file, open a network connection, start a process). A crash in a program doesn't take down the whole system; a crash in the kernel does.

Processes, threads and memory

Concept	Meaning
Program	Instructions stored in a file
Process	A running program with its own memory space and an ID (PID)
Thread	A unit of execution inside a process; threads share the process memory
Virtual memory	Each process sees its own address space; the OS maps it to physical RAM or swap
Context switch	The OS pausing one task and running another, many times per second

Common operating systems

OS	Where
Linux (Ubuntu, RHEL, Amazon Linux, Debian, Alpine)	Servers, cloud, containers, Kubernetes, embedded, Android
Windows / Windows Server	Office PCs, Active Directory, .NET apps, some enterprise servers
macOS	Developer laptops
Unix (AIX, Solaris)	Legacy enterprise systems

Boot process (any computer)

Power on → firmware (BIOS/UEFI) checks hardware (POST) → bootloader loads the OS kernel → kernel initializes hardware and starts the first process → system services start → login screen/prompt.

1.5 Software

Type	Examples
System software	OS, drivers, firmware
Application software	Browsers, databases, web servers, your company's app
Middleware	Message queues, application servers (Tomcat), API gateways
Utilities / tools	Git, Docker, Terraform, kubectl

How programs are built and run

Model	How	Examples
Compiled	Source code → compiler → machine-code binary for one OS/architecture	C, C++, Go, Rust
Interpreted	An interpreter reads and runs the code	Python, Bash, Ruby, JavaScript (Node.js)
Bytecode + VM	Compiled to bytecode, run by a virtual machine	Java (JVM), C# (.NET)

Dependencies are libraries a program needs. Managing them (versions, conflicts) is a big part of DevOps; containers solve much of this by packaging the app with its dependencies.

Open source and licenses

Open-source software has publicly available source code (Linux, Kubernetes, Terraform/OpenTofu, Python). Licenses decide how you may use it: **permissive** (MIT, Apache 2.0, BSD: use freely, keep notices) vs **copyleft** (GPL: modified versions you distribute must stay open). Companies check licenses before using libraries.

Versions

Semantic versioning `MAJOR.MINOR.PATCH` (e.g. `2.7.1`): major = breaking changes, minor = new features, patch = bug fixes. **LTS** = long-term support release.

1.6 Files, folders and paths

	Windows	Linux / macOS
Root	Drive letters `C:\`	One tree starting at `/`
Separator	`\`	`/`
Home	`C:\Users\thirupathi`	`/home/thirupathi` (`~`)
Case sensitive	No	**Yes** (`File.txt` ≠ `file.txt`)
Hidden files	Attribute	Name starts with `.`
Executable	By extension (`.exe`, `.bat`)	By **permission** (`chmod +x`)

Absolute path = full path from the root. **Relative path** = from the current folder. **File extension** is only a hint (`file` command shows the real type on Linux).

1.7 The command line

A **CLI (command-line interface)** lets you control a computer by typing commands; a **GUI** uses windows and clicks.

Why professionals use the CLI: servers have no GUI; commands can be **scripted, automated, repeated and version-controlled**; it's faster and precise; remote access over SSH is text-based; every DevOps tool (git, docker, kubectl, terraform, aws) is a CLI.

Term	Meaning
Terminal	The window/app showing a text session (Windows Terminal, GNOME Terminal)
Shell	The program interpreting commands (PowerShell, cmd, bash, zsh)
Prompt	Text showing the shell is ready (`PS C:\Users\me>` , `user@host:~$`)
Command / options / arguments	`ls -l /var/log` → command `ls` , option `-l` , argument `/var/log`

Windows PowerShell essentials

Task	PowerShell	Linux equivalent
Where am I	`Get-Location` (`pwd`)	`pwd`
List files	`Get-ChildItem` (`ls` , `dir`)	`ls`
Change folder	`Set-Location` (`cd`)	`cd`
Make folder	`New-Item -ItemType Directory name` (`mkdir`)	`mkdir`
Read file	`Get-Content file` (`cat`)	`cat`
Copy / move / delete	`Copy-Item` , `Move-Item` , `Remove-Item`	`cp` , `mv` , `rm`
Computer name / user	`hostname` , `whoami`	`hostname` , `whoami`
Network config	`ipconfig /all` , `Get-NetIPAddress`	`ip a`
Test reachability / port	`ping host` , `Test-NetConnection host -Port 443`	`ping` , `nc -zv`
DNS lookup	`Resolve-DnsName name` , `nslookup`	`dig` , `nslookup`
Processes	`Get-Process` , `Stop-Process -Id 123`	`ps` , `kill`
Services	`Get-Service` , `Restart-Service name`	`systemctl`
Help	`Get-Help Get-Process -Examples`	`man`

WSL2 (Windows Subsystem for Linux) runs a real Linux kernel inside Windows: `wsl --install` (admin PowerShell), then use Ubuntu like a Linux server. This is the recommended setup for learning DevOps on Windows.

1.8 Client–server model and how the web works



Term	Meaning
Client	Requests a service
Server	Provides a service; "server" is a **role** (software listening on a port), usually on a dedicated machine
Request / response	The basic exchange (HTTP, database queries, API calls)
Port	Number identifying the service on a machine (443 HTTPS, 22 SSH, 5432 PostgreSQL)
API	A defined way for programs to talk to each other (often HTTP + JSON)
Stateless vs stateful	Stateless servers keep no session data between requests (easy to scale); stateful ones do (databases)

Common architectures

Architecture	Description
Monolith	One application containing all features; simple to start, harder to scale teams
3-tier	Presentation (web) → application (logic) → data (database)
Microservices	Many small services communicating over APIs; independent deployment and scaling, more operational complexity
Serverless	Functions run on demand by a cloud provider (AWS Lambda)

1.9 Networks and the internet (overview)

Term	Meaning
Network	Connected devices sharing data
LAN / WAN	Local network (home, office) / wide area network (connects sites; the internet is the biggest)
IP address	Logical address of a device (`192.168.1.10`)
MAC address	Hardware address of a network card
Router / switch	Connects networks / connects devices inside one network
DNS	Translates names to IP addresses
ISP	Internet service provider
Bandwidth / latency	Capacity (Mbps) / delay (ms)

Full detail: **Networking Basics** notes (topic 2) and the **Computer Networking** book.

1.10 Data centers, virtualization and cloud

Data centers

Buildings full of **racks** of servers, storage and network equipment, with redundant power (UPS, generators), cooling, physical security and multiple internet connections. Companies either run their own (**on-premises**), rent space (**colocation**), or use **cloud** providers who run massive data centers for them.

Virtualization

```
Without virtualization:   1 physical server → 1 OS → apps (often 10–20% used)
With virtualization:      1 physical server → HYPERVISOR → many VMs, each with its own OS
```

Term	Meaning
Hypervisor	Software that runs virtual machines. **Type 1** runs directly on hardware (VMware ESXi, KVM, Hyper-V, AWS Nitro); **Type 2** runs on a normal OS (VirtualBox, VMware Workstation)
Virtual machine (VM)	A complete simulated computer with its own OS
Host / guest	Physical machine / VM running on it
Snapshot	Saved state of a VM

Benefits: better hardware utilization, isolation, fast provisioning, snapshots, live migration. Every cloud instance (EC2) is a VM.

Containers

Containers package an app with its libraries and share the **host's kernel**: lighter and faster than VMs (MBs vs GBs, seconds vs minutes). Docker builds and runs them; Kubernetes runs them at scale. Covered fully in the Docker and Kubernetes notes.

	VM	Container
Includes	Full guest OS	App + dependencies
Isolation	Strong (own kernel)	Process-level (shared kernel)
Size / start	GBs / minutes	MBs / seconds

Cloud computing

Renting compute, storage, networking and services on demand over the internet, paying per use. Models: **IaaS** (VMs, networks), **PaaS** (managed platforms/databases), **SaaS** (finished software). Providers: AWS, Microsoft Azure, Google Cloud. Covered fully in the AWS notes.

1.11 Security fundamentals

Concept	Meaning
CIA triad	**Confidentiality** (only authorized people see data), **Integrity** (data isn't altered), **Availability** (systems are usable when needed)

Concept	Meaning
Authentication	Proving who you are (password, key, certificate)
Authorization	What you are allowed to do
MFA	Two or more factors: something you know (password), have (phone/key), are (fingerprint)
Least privilege	Give only the access needed, nothing more
Principle of defense in depth	Multiple layers of protection
Patch	Update that fixes bugs/vulnerabilities; patching is a core ops task
Vulnerability / CVE	A weakness / a public ID for a known vulnerability (CVE-2024-xxxx)

Encryption basics

Type	How	Used in
Symmetric	Same secret key encrypts and decrypts (AES)	Disk encryption, bulk data, inside TLS sessions
Asymmetric	Key **pair**: public key encrypts/verifies, private key decrypts/signs (RSA, ECDSA, Ed25519)	SSH keys, TLS certificates, digital signatures
Hashing	One-way fingerprint of data (SHA-256); can't be reversed	Password storage (with salt, bcrypt/argon2), file integrity checksums, Git commit IDs
Encoding (not security!)	Reversible format change (Base64)	Transport of binary data; Kubernetes Secrets are only Base64 by default

Golden rules: never share private keys or passwords, never commit secrets to Git, use a password manager and MFA, keep systems patched.

1.12 IT roles and how they work together

Role	Focus
Software developer	Builds application features
QA / test engineer	Tests quality, automates tests
System administrator	Manages servers and OS
Network engineer	Designs and runs networks
Database administrator (DBA)	Databases: performance, backups, security
Security engineer	Protects systems and data
DevOps engineer	Automates build, test, deploy and infrastructure; bridges dev and ops
Cloud engineer	Designs and operates cloud infrastructure
SRE	Reliability through engineering: SLOs, monitoring, incident response
Platform engineer	Builds internal platforms and self-service tools for developers
Cloud support engineer	Helps customers troubleshoot cloud issues

Software delivery lifecycle (SDLC): plan → code → build → test → release → deploy → operate → monitor → feedback → plan. DevOps automates and connects these steps.

1.13 Basic troubleshooting mindset

1. **Reproduce and define:** what exactly fails, for whom, since when? Get the exact error message.
2. **What changed?** Updates, deployments, config changes, network changes.
3. **Check the simple things first:** power, cable/Wi-Fi, disk space, is the service running, correct credentials.
4. **Divide and conquer:** split the system into layers (network → OS → service → app → data) and test each.
5. **One change at a time,** then test again.
6. **Read logs and error messages carefully:** search the exact message.
7. **Document** what you found and fixed.

1.14 Interview questions and answers

#	Question	Answer
1	RAM vs storage?	RAM is fast temporary working memory, cleared at power off; storage is slower and permanent
2	What is a vCPU in the cloud?	Usually one hardware thread of a physical core
3	HDD vs SSD vs NVMe?	Spinning disk (slow, cheap) / flash (fast) / flash over PCIe (fastest)
4	IOPS vs throughput?	Operations per second (small random I/O) vs MB/s (large sequential I/O)
5	What does an operating system do?	Manages processes, memory, files, devices, networking, security; gives programs an interface to hardware
6	Kernel mode vs user mode?	Kernel has full hardware access; programs run restricted and use system calls
7	Process vs thread?	A process has its own memory; threads inside a process share memory
8	What is virtual memory / swap?	Each process gets its own address space; swap uses disk when RAM is full
9	Compiled vs interpreted languages?	Compiled to machine code ahead of time vs executed by an interpreter at runtime
10	What is a hypervisor? Type 1 vs 2?	Software running VMs; type 1 on bare metal, type 2 on a host OS
11	VM vs container?	VM has its own OS kernel; container shares the host kernel and is lighter
12	What is the client-server model?	Clients send requests; servers listen on ports and respond
13	Why use the CLI?	Automation, remote access, precision, speed; all DevOps tools are CLIs
14	Explain the CIA triad	Confidentiality, integrity, availability: the three goals of security
15	Symmetric vs asymmetric encryption?	One shared key vs public/private key pair
16	Hashing vs encryption vs encoding?	One-way fingerprint / reversible with a key / reversible format change with no security

#	Question	Answer
17	What is MFA?	Authentication with two or more different factor types
18	What is semantic versioning?	MAJOR.MINOR.PATCH: breaking / feature / fix
19	Why are Linux paths case-sensitive problems common?	`Config.yaml` and `config.yaml` are different files on Linux but not on Windows
20	What is x86-64 vs ARM64 and why does it matter?	Different CPU architectures; binaries and container images must match the architecture

1.15 Self-check: I can…

- [ ] Explain CPU (cores, threads, architecture), RAM, storage types and their performance terms.
- [ ] Convert between bits and bytes and explain GB vs GiB and Mbps vs MB/s.
- [ ] Explain what an OS does, kernel vs user mode, processes, threads, virtual memory.
- [ ] Explain compiled vs interpreted software, dependencies, open-source licenses and versioning.
- [ ] Use basic PowerShell commands and explain why WSL2 is useful.
- [ ] Describe the client-server model and the steps of loading a website.
- [ ] Explain data centers, virtualization (hypervisors, VMs), containers and cloud models.
- [ ] Explain the CIA triad, authentication vs authorization, MFA, encryption, hashing and encoding.
- [ ] Name the main IT roles and how DevOps, Cloud, SRE and Platform engineers fit in.
- [ ] Apply a structured troubleshooting approach.

REMEMBER

- Every computer: **Input → Process → Store → Output**.
- RAM is fast but forgets at power-off; storage keeps data.
- Servers usually run **Linux**, have no screen, and are managed through a **terminal** over a network.